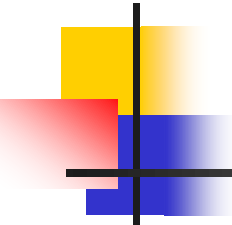


Java DataBase Connectivity

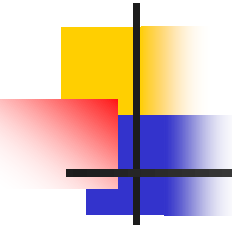


Dr. Khadim Dramé
Khadim.drame@univ-zig.sn
Département Informatique
Université Assane Seck de Ziguinchor



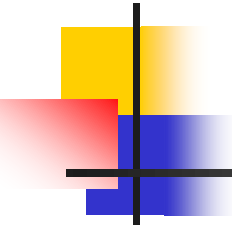
Plan du cours

- Introduction
- Interfaces et classes de JDBC
- Connexion à une base de données
- Manipulation de requêtes SQL simples
- Manipulation de requêtes SQL paramétrées
- Traitement des résultats
- Réorganisation du code



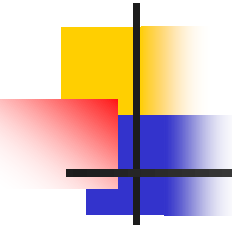
Introduction

- JDBC, API de Java SE
 - pour l'accès aux bases de données relationnelles avec le langage SQL
 - indépendante d'un SGBD
 - nécessitant un pilote (ou **driver**) spécifique au SGBD, fourni par le propriétaire
 - supportant les différents types de requêtes SQL
 - fournie par le package **java.sql**



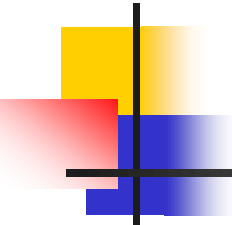
Interfaces de JDBC

- Interfaces principales de `java.sql`
 - `Driver` : chargement et configuration du driver
 - `Connection` : connexion à une base de données
 - `Statement` : requête SQL
 - `PreparedStatement` : requête SQL paramétrée
 - `CallableStatement` : procédure stockée sur le SGBD
 - `ResultSet` : contient les lignes retournées par un Select
 - `ResultSetMetaData` : description des lignes récupérées par un SELECT
 - `DatabaseMetaData` : informations sur la base de données



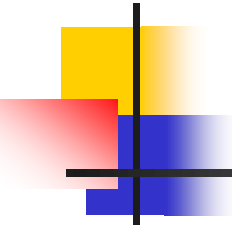
Drivers (pilotes)

- L'utilisation des interfaces de JDBC requiert des classes les implémentant
- Ces classes sont fournies par un **driver JDBC**
- Un driver est spécifique à un SGBD
- La plupart des SGBDs disposent de drivers JDBC, fournis par leurs éditeurs



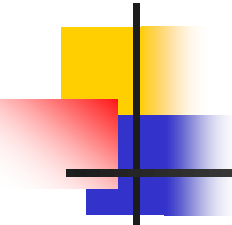
Étapes pour interagir avec une BD

1. Charger le driver (**inutilisé depuis JDK 6**)
2. Établir une connexion (**Connection**) avec la BD
3. Créer des requêtes SQL (**Statement**, **PreparedStatement** ou **CallableStatement**)
4. Lancer l'exécution des requêtes (**executeQuery()**, **executeUpdate()**) et traiter le type de retour
5. Fermer la connexion (**close()**)



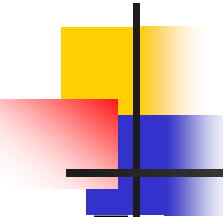
Connexion à une base de données

- Pour établir une connexion à une BD, on utilise la méthode statique `getConnection()` de `DriverManager`
- `getConnection()`
 - prend en paramètres une `URL` et d'autres informations (`nom d'utilisateur`, `mot de passe`, ...)
 - renvoie un objet de type `Connection` si la connexion est établie
 - renvoie `null` si la connexion échoue
- L'instance de `Connection` permet de lancer des requêtes SQL au SGBD



Connexion à une base de donnée

- Pour établir une connexion à une BD
 - spécifier l'URL: `jdbc:mysql://hote:port/nombd`
 - **hote**: nom de l'hôte sur lequel le serveur est installé (localhost ou 127.0.0.1 par exemple)
 - **port**: port utilisé par le serveur (3306 par défaut)
 - **nombd**: nom de la BD
 - spécifier le nom de l'utilisateur et son mot de passe
- On peut aussi spécifier d'autres paramètres
`jdbc:mysql://hote:port/nombd?useSSL=false&serverTimezone=UTC`
- NB: La forme de l'URL dépend du SGBD

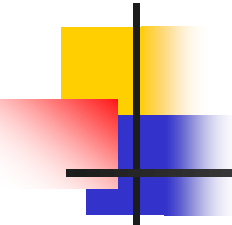


Connexion à une base de données

■ Exemple

```
String url = "jdbc:mysql://localhost:3306/club?serverTimezone=UTC";  
//String url = "jdbc:postgresql://localhost:5432/club";  
String user = "root";  
String password = "";  
Connection co = null;  
try {  
    co = DriverManager.getConnection(url, user, password);  
    if (co != null) {  
        System.out.println("Connexion réussie !");  
    } else {  
        System.out.println("Echec de le connexion!");  
    }  
} catch (SQLException e) {  
    e.printStackTrace();  
}
```

■ NB: recommandation d'utiliser try-with-resources



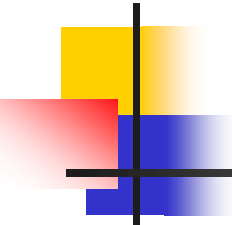
Manipulation de requêtes SQL

- Création de requêtes SQL simples
 - On utilise la méthode `createStatement()` de **Connection**

```
Statement st = co.createStatement();
```

- Un **statement** créé peut être utilisé pour lancer plusieurs requêtes différentes
- Exécution de requêtes SQL de consultation (select)
 - On utilise la méthode `executeQuery()` de **Statement**
 - Cette méthode renvoie un **ResultSet**

```
ResultSet rs = st.executeQuery("SELECT * FROM MEMBRE");
```

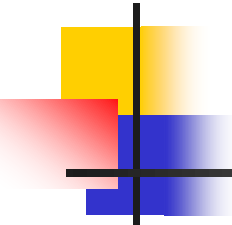


Manipulation de requêtes SQL

- Exécution de requêtes de modification (insert, update, delete)
 - On utilise la méthode `executeUpdate()` de `Statement`
 - Cette méthode renvoie le nombre de lignes modifiées

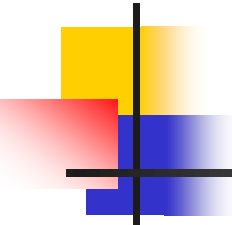
```
int nbL = st.executeUpdate("INSERT INTO MEMBRE VALUES  
(5, 'Modou', 'Diop', 42)");
```

- Modification de tables et vues (create table, drop table,...)
 - On utilise la méthode `executeUpdate()` de `Statement`
- Si on ne connaît pas la nature de l'ordre SQL: `execute()`



Requêtes SQL paramétrées

- Pour créer des requêtes paramétrées («à trous»), on utilise la méthode `prepareStatement()` de **Connection**
- La méthode `prepareStatement()` renvoie un objet de type **PreparedStatement**
- Elles peuvent être exécutées avec plusieurs ensembles de valeurs
- Les valeurs des paramètres sont spécifiés avec des méthodes `setXXX` (`setInt`, `setString`, `setDouble`)

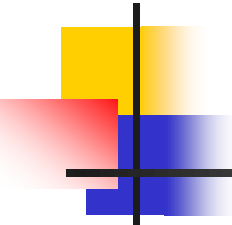


Requêtes SQL paramétrées

■ Exemples

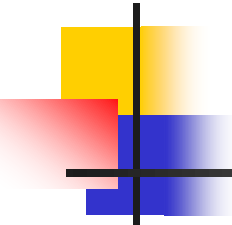
```
String req = "SELECT * FROM MEMBRE WHERE prenom=? AND age>?";  
PreparedStatement pst = co.prepareStatement(req);  
pst.setString(1, "Moustapha");  
pst.setInt(2, 35);  
ResultSet rs = pst.executeQuery();
```

```
String req = "INSERT INTO MEMBRE (nom, prenom) VALUES (?, ?)";  
PreparedStatement pst = co.prepareStatement(req);  
pst.setString(1, "FALL");  
pst.setString(2, "Moussa");  
pst.executeUpdate();
```



Avantages des requêtes paramétrées

- Leur traitement est plus **rapide** si elles sont utilisées avec des paramètres différents
- Elles améliorent la **portabilité** (methodes setXXX gèrent les différences entre SGBD)
- Elles permettent d'éviter l'injection de codes SQL



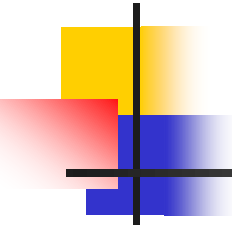
Injection de codes SQL

- Faible de sécurité qu'un utilisateur malveillant peut exploiter en injectant une requête SQL non prévue
- Elle peut compromettre la sécurité de l'application

```
String req = "SELECT * FROM USER WHERE login = '" + login + "'  
and password = '" + password + "'";
```

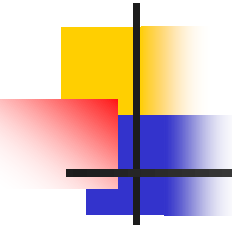
```
Login: ' or '' = '  
Password: ' or '' = ';
```

```
String req = "SELECT * FROM USER WHERE login = ' or '' = ' and  
password = ' or '' = '";
```



Traitement des résultats des requêtes

- **ResultSet** permet de parcourir toutes les lignes renvoyées par un **select**, une à une
- Sa méthode **next()**
 - permet de passer à la ligne suivante
 - renvoie **true** si cette ligne existe
 - renvoie **false** sinon
- Ses méthodes **getXxx()** permettent de récupérer les valeurs des colonnes par leur **index** ou leur **nom**
 - **Xxx** désigne le type java correspondant
`getInt(1), getString(2),`
`getString("nom"), getInt("age")`

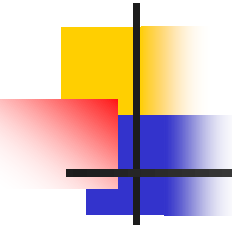


Traitement des résultats des requêtes

- Récupération des données (select)
 - `executeQuery()` renvoie un **ResultSet** permettant de récupérer les lignes une à une

```
// Récupère les lignes une à une
while(rs.next()){
    String pnom = rs.getString("prenom");//ou rs.getString(2);
    String nom = rs.getString("nom");
    int age = rs.getInt("age");
    System.out.println(pnom+" "+nom+" a "+age+" ans");
}
```

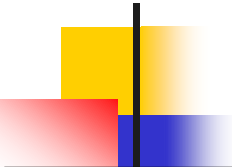
- On peut utiliser le nom ou l'index de la colonne



Traitement des résultats des requêtes

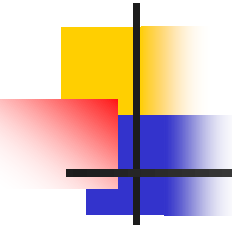
- Une requête de mise à jour (`executeUpdate()`) renvoie
 - 0 en cas d'échec de la mise à jour
 - le nombre de lignes mises à jour sinon

```
int nbL = st.executeUpdate("INSERT INTO MEMBRE VALUES  
(6, 'Modou', 'Diop', 42)");  
if (nbL != 0)  
    System.out.println("insertion réussie");  
else  
    System.out.println("problème d'insertion");
```



Exemple complet

```
String url = "jdbc:mysql://localhost:3306/club?serverTimezone=UTC";
//String url = "jdbc:postgresql://localhost:5432/club";
String user = "root";
String mdp = "";
Connection co = null;
Statement st = null;
try{
    co = DriverManager.getConnection(url, login, password);
    st = co.createStatement();
    ResultSet rs = st.executeQuery("SELECT * FROM MEMBRE");
    while(rs.next()){
        String pnom = rs.getString("prenom");//ou rs.getString(2);
        String nom = rs.getString("nom");
        int age = rs.getInt("age");
        System.out.println(pnom+" "+nom+" a "+age+" ans");
    }
}catch (SQLException e) {
    e.printStackTrace();
}finally{
    rs.close();
    st.close();
    con.close();
}
```

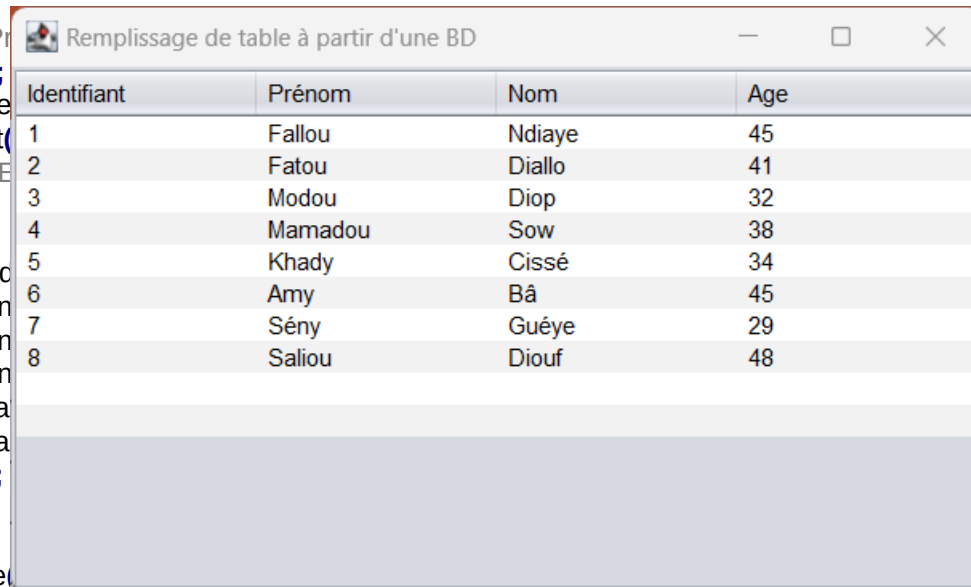


Exemple complet (try with resources)

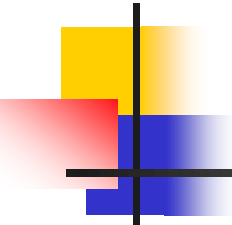
```
String url = "jdbc:mysql://localhost:3306/club?serverTimezone=UTC";  
//String url = "jdbc:postgresql://localhost:5432/club";  
String user = "root";  
String mdp = "";  
try(Connection co = DriverManager.getConnection(url, login, mdp);  
    Statement st = co.createStatement();  
    ResultSet rs = st.executeQuery("SELECT * FROM MEMBRE")){  
    while(rs.next()){  
        String pnom = rs.getString("prenom");//ou rs.getString(2);  
        String nom = rs.getString("nom");  
        int age = rs.getInt("age");  
        System.out.println(pnom+" "+nom+" a "+age+" ans");  
    }  
}catch (SQLException e) {  
    e.printStackTrace();  
}
```

Remplir une table à partir d'une BD

```
public static void main(String[] args) throws UnsupportedLookAndFeelException {
    UIManager.setLookAndFeel(new NimbusLookAndFeel());
    JFrame f = new JFrame("Remplissage de table à partir d'une BD");
    //paramètres de connexion
    String url = "jdbc:mysql://localhost:3306/club?serverTimezone=UTC";
    String user = "root";
    String password = "";
    String columns[] = {"Identifiant", "Prénom", "Nom", "Age"};
    String data[][] = new String[10][4];
    try (Connection co = DriverManager.getConnection(url, user, password);
        Statement st = co.createStatement();
        ResultSet rs = st.executeQuery("SELECT * FROM club")) {
        int i = 0;
        while (rs.next()) {
            int id = rs.getInt("Identifiant");
            String prenom = rs.getString("Prénom");
            String nom = rs.getString("Nom");
            String age = rs.getString("Age");
            data[i][0] = id;
            data[i][1] = prenom;
            data[i][2] = nom;
            data[i][3] = age;
            i++;
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    DefaultTableModel model = new DefaultTableModel(data, columns);
    JTable table = new JTable(model);
    JScrollPane pane = new JScrollPane(table);
    f.add(pane);
    f.setSize(500, 300);
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    f.setVisible(true);
}
```



Identifiant	Prénom	Nom	Age
1	Fallou	Ndiaye	45
2	Fatou	Diallo	41
3	Modou	Diop	32
4	Mamadou	Sow	38
5	Khady	Cissé	34
6	Amy	Bâ	45
7	Sény	Guéye	29
8	Saliou	Diouf	48

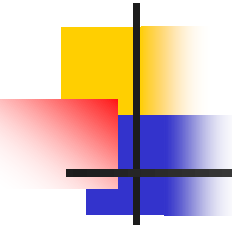


Réorganisation du code

- Mettre toutes les données relatives à la connexion ([url](#), [nom utilisateur](#), [mot de passe](#), ...) dans une classe Java
- Pour chaque table de la BD, créer une classe Java ayant comme attributs les colonnes de cette table
- Mettre le code correspondant à l'accès aux données dans de nouvelles classes et interfaces: elles constituent la couche **DAO** (Data Acces Object)

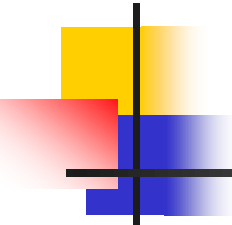
Classe Connexion

```
package sn.uasz.m1.bd;
public class Connexion{
    private static String url = "jdbc:mysql://localhost:3306/club?serverTimezone=UTC";
    private static String user = "root";
    private static String mdp = "";
    private static Connection co = null;
    public static Connection getCo(){
        try {
            co = DriverManager.getConnection(url, user, mdp);
        } catch (Exception e){
            e.printStackTrace();
        }
        return co;
    }
    public static boolean close(){
        if (co != null) {
            try {
                co.close();
                return true;
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
        return false;
    }
}
```



Classe **Membre**

- Attributs: identifiant, prénom, nom, âge
- Getters et setters de ces différents attributs
- Constructeurs sans et avec paramètres



Interface IDao

```
package sn.uasz.m1.bd;
import java.util.List;
public interface IDao<T> {
    boolean create(T o);
    boolean update(Membre o);
    boolean delete(T o);
    T findById(int id);
    List<T> getAll();
}
```

- Toutes ces méthodes devront être implémentées

Classe MembreImpl

```
public class MembreImp implements IDao<Membre> {
    @Override
    public boolean create(Membre o) {
        try {
            String req = "insert into membre values(" + o.getId() + "," + o.getPrenom() + "," + o.getNom() + "," + o.getAge();
            Statement st = Connexion.getCo().createStatement();
            if (st.executeUpdate(req) == 1) {
                return true;
            }
        } catch (SQLException ex) {
            System.out.println("Erreur de requête SQL");
        }
        return false;
    }

    @Override
    public boolean delete(Membre o) {
        try {
            String req = "delete from membre where id=" + o.getId();
            Statement st = Connexion.getCo().createStatement();
            if (st.executeUpdate(req) == 1) {
                return true;
            }
        } catch (SQLException ex) {
            System.out.println("Erreur de requête SQL");
        }
        return false;
    }
}
```

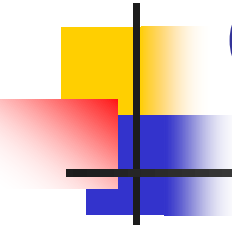
Classe MembreImpl

@Override

```
public boolean update(Membre o) {
    try {
        String req = "update membre set nom = '"+o.getNom()+"', prenom = '"+o.getPrenom()+"' where id = '"+o.getId();
        Statement st = Connexion.getCo().createStatement();
        if(st.executeUpdate(req) == 1) {
            return true;
        }
    } catch (SQLException ex) {
        System.out.println("Erreur de requête SQL");
    }
    return false;
}
```

@Override

```
public Membre findById(int id) {
    try {
        String req = "select * from membre where id=" + id;
        Statement st = Connexion.getCo().createStatement();
        ResultSet rs = st.executeQuery(req);
        if (rs.next()) {
            return new Membre(rs.getInt("id"), rs.getString("prenom"), rs.getString("nom"), rs.getInt("age"));
        }
    } catch (SQLException ex) {
        System.out.println("Erreur de requête SQL");
    }
    return null;
}
```



Classe MembreImpl

@Override

```
public List<Membre> findAll() {  
    List<Membre> list = new ArrayList<Membre>();  
    try {  
        String req = "select * from membre";  
        Statement st = Connexion.getCo().createStatement();  
        ResultSet rs = st.executeQuery(req);  
        while (rs.next()) {  
            list.add(new Membre(rs.getInt("id"), rs.getString("prenom"), rs.getString("nom"), rs.getInt("age")));  
        }  
    } catch (SQLException ex) {  
        System.out.println("Erreur de requête SQL");  
    }  
    return list;  
}
```